

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ  
федеральное государственное автономное образовательное учреждение высшего образования  
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ  
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

КАФЕДРА № 41

ОТЧЕТ О НАУЧНО-ИССЛЕДОВАТЕЛЬСКОЙ РАБОТЕ

ИССЛЕДОВАНИЕ ИНСТРУМЕНТА PYSPARK ДЛЯ РАСПРЕДЕЛЕННОЙ ОБРАБОТКИ  
БОЛЬШИХ ДАННЫХ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. №

4217

подпись, дата

У. А. Мазориев

инициалы, фамилия

Санкт-Петербург 2026

## СОДЕРЖАНИЕ

|                                                                        |    |
|------------------------------------------------------------------------|----|
| ВВЕДЕНИЕ .....                                                         | 4  |
| 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ .....                                     | 7  |
| 1.1. Современное состояние систем обработки больших данных .....       | 7  |
| 1.2. Проблемы и вызовы распределённой аналитики .....                  | 8  |
| 1.3. Место PySpark в современном data stack.....                       | 8  |
| 2. НАУЧНЫЕ ОСНОВЫ .....                                                | 10 |
| 2.1. Распределённые вычисления и обработка в памяти.....               | 10 |
| 2.2. Модель RDD и механизм lineage.....                                | 10 |
| 2.3. DataFrame, Spark SQL и оптимизация запросов .....                 | 11 |
| 2.4. Узкие и широкие преобразования, shuffle и партиционирование ..... | 12 |
| 2.5. Надёжность, кэширование и контрольные точки .....                 | 12 |
| 3. ИССЛЕДОВАНИЕ ИНСТРУМЕНТА PYSPARK.....                               | 14 |
| 3.1. Архитектура Spark-приложения .....                                | 14 |
| 3.2. Основные компоненты и абстракции.....                             | 14 |
| 3.3. Работа со схемами и сложными типами данных .....                  | 15 |
| 3.4. Форматы данных и интеграции .....                                 | 16 |
| 4. ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ ОБРАБОТКИ ДАННЫХ.....                       | 17 |
| 4.1. Реализация аналитики на RDD .....                                 | 18 |
| 4.2. Реализация аналитики на DataFrame.....                            | 19 |
| 4.3. Паттерны производительности и качества .....                      | 20 |
| 5. ПРЕИМУЩЕСТВА И ОГРАНИЧЕНИЯ PYSPARK .....                            | 22 |
| 6. СРАВНЕНИЕ С ДРУГИМИ ИНСТРУМЕНТАМИ.....                              | 24 |

|                                                     |    |
|-----------------------------------------------------|----|
| 7. СОВМЕСТИМОСТЬ PYSPARK С ЭКОСИСТЕМОЙ ДАННЫХ ..... | 26 |
| ЗАКЛЮЧЕНИЕ .....                                    | 28 |
| СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ .....              | 30 |

## ВВЕДЕНИЕ

В условиях цифровой трансформации и роста объёмов данных всё большее значение приобретают инструменты распределённой обработки. Информационные системы формируют журналы событий, транзакционные записи, телеметрию, пользовательские действия и справочные наборы данных, которые необходимо обрабатывать быстрее, надёжнее и воспроизводимее, чем это возможно в рамках одного вычислительного узла.

Apache Spark является одной из наиболее распространённых платформ для пакетной и интерактивной обработки больших данных. Его модель выполнения основана на распределении вычислений между узлами кластера, хранении промежуточных данных в памяти, построении графа зависимостей и автоматическом восстановлении вычислений при сбоях. PySpark предоставляет доступ к этим возможностям из языка Python, что делает инструмент удобным для аналитиков данных, дата-инженеров и исследователей.

Актуальность настоящей работы обусловлена необходимостью системного анализа PySpark как инструмента, объединяющего низкоуровневые механизмы RDD и высокоуровневые API DataFrame/Spark SQL. В практических проектах важно не только уметь запустить вычисление, но и понимать, как формируются партиции, когда возникает shuffle, чем опасны операции на драйвере, какие преимущества даёт схема DataFrame и каким образом оптимизатор Spark влияет на производительность.

Проблема исследования заключается в противоречии между ростом объёмов и разнообразия данных, с одной стороны, и ограничениями традиционных локальных подходов к аналитике, с другой. Инструменты вроде pandas удобны для быстрых исследований, но при увеличении объёма данных сталкиваются с ограничениями памяти и масштабирования. PySpark решает эти

проблемы, однако требует понимания распределённой модели исполнения и дисциплины в проектировании преобразований.

Целью данной работы является исследование архитектурных, методологических и практических особенностей PySpark, а также оценка применимости инструмента для обработки больших данных на основе выполненной практики с RDD и DataFrame.

Для достижения поставленной цели в работе решаются следующие задачи:

1. Проанализировать современное состояние систем распределённой обработки данных.
2. Рассмотреть научные основы Apache Spark: вычисления в памяти, DAG, партиционирование, отказоустойчивость и ленивое выполнение.
3. Изучить ключевые абстракции PySpark: SparkSession, RDD, DataFrame, Spark SQL, transformations и actions.
4. Исследовать практические сценарии обработки данных с использованием RDD API и DataFrame API.
5. Оценить преимущества, ограничения и риски применения PySpark при работе с большими данными.
6. Выполнить сравнительный анализ PySpark с альтернативными инструментами обработки данных.
7. Сформулировать выводы о практической значимости PySpark и направлениях дальнейшего развития проекта.

Объектом исследования являются системы распределённой обработки и анализа данных. Предметом исследования являются методы, архитектурные механизмы и программные интерфейсы PySpark, применяемые для построения аналитических вычислений над большими наборами данных.

Научная новизна работы состоит в комплексном рассмотрении PySpark сразу на двух уровнях абстракции: RDD как фундаментальной модели распределённых наборов данных и DataFrame как оптимизируемого табличного API. Практическая значимость заключается в том, что результаты работы могут быть использованы как методическая основа для разработки учебных и прикладных проектов по анализу больших данных.

Структурно работа состоит из семи глав. В первой главе рассматривается предметная область больших данных и место PySpark в современных аналитических системах. Во второй главе раскрываются научные основы Spark. Третья глава посвящена исследованию инструмента PySpark. В четвёртой главе описана практическая реализация обработки данных. В пятой главе сформулированы преимущества и ограничения инструмента. В шестой главе выполнено сравнение с альтернативами. В седьмой главе рассмотрена совместимость PySpark с экосистемой обработки данных.

# **1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ**

## **1.1. Современное состояние систем обработки больших данных**

Современная аналитика данных развивается в условиях постоянного роста объёмов информации, скорости её поступления и разнообразия форматов. Организации используют события веб-приложений, финансовые показатели, логи серверов, телеметрию устройств, файлы в форматах CSV, JSON, Parquet и данные из реляционных или NoSQL-хранилищ. Такие данные необходимо не только хранить, но и регулярно преобразовывать, агрегировать, проверять и передавать в аналитические витрины.

Классический подход к обработке данных на одном сервере становится недостаточным при увеличении объёма исходных наборов. Основные ограничения связаны с объёмом оперативной памяти, временем чтения и записи на диск, невозможностью параллельной обработки на нескольких узлах и трудностью повторного выполнения вычислений при сбоях. Поэтому в системах больших данных используется горизонтальное масштабирование: данные делятся на части, а вычисления распределяются между рабочими узлами.

Apache Spark стал развитием идей распределённой обработки, предложенных в экосистеме Hadoop, но сделал акцент на вычислениях в памяти и более выразительных API. В отличие от классического MapReduce, где каждый этап часто материализуется на диск, Spark стремится удерживать промежуточные данные в RAM и строить целостный граф вычислений. Это особенно важно для итеративных алгоритмов, интерактивной аналитики и конвейеров с несколькими связанными преобразованиями.

## **1.2. Проблемы и вызовы распределённой аналитики**

Распределённая обработка данных решает проблему масштаба, но создаёт новые инженерные вызовы. Разработчику необходимо учитывать размещение данных, количество партиций, стоимость передачи данных по сети, наличие перекоса ключей, объём данных, собираемых на драйвере, и влияние широких преобразований на shuffle. Неправильно выбранная операция может превратить распределённое вычисление в узкое место, например при использовании `collect()` для большого набора данных.

Важной проблемой является качество данных. Сырые наборы могут содержать пропуски, некорректные типы, дубликаты, разнородные форматы дат, вложенные структуры и повреждённые строки. В практической части работы эта проблема проявляется при обработке логов сенсоров с NULL-значениями, CSV-файлов с финансовыми показателями, Parquet-таблиц с дубликатами и строковых списков, которые необходимо преобразовать в массивы.

Отдельное значение имеет наблюдаемость выполнения. Для эффективной работы с большими данными требуется понимать физический план запроса, видеть операции `Exchange`, `HashAggregate`, `SortMergeJoin`, `BroadcastHashJoin` и оценивать, где происходит перераспределение данных. Поэтому PySpark следует рассматривать не только как библиотеку Python, но и как интерфейс к распределённому вычислительному движку.

## **1.3. Место PySpark в современном data stack**

PySpark занимает промежуточное положение между языком прикладной аналитики и вычислительным ядром Apache Spark. С одной стороны, разработчик пишет код на Python и использует привычные конструкции языка. С другой стороны, фактическое выполнение происходит в JVM-процессах



Spark, а Python-код взаимодействует с ними через Py4J и внутренние механизмы сериализации.

В современном data stack PySpark применяется в ETL/ELT-конвейерах, ad hoc-аналитике, подготовке признаков для машинного обучения, обработке журналов, миграции данных между форматами и построении аналитических витрин. Инструмент интегрируется с файловыми хранилищами, облачными платформами, оркестраторами, системами контроля версий, ноутбуками и BI-инструментами.

Практическая ценность PySpark особенно заметна в учебных и исследовательских проектах: один и тот же инструмент позволяет сначала разобрать фундаментальную модель RDD, затем перейти к DataFrame API, оконным функциям, joins, вложенным типам данных и оптимизированному чтению файловых форматов.

## **2. НАУЧНЫЕ ОСНОВЫ**

### **2.1. Распределённые вычисления и обработка в памяти**

Распределённые вычисления основаны на разбиении задачи на множество независимых или частично зависимых подзадач, которые выполняются параллельно на разных узлах. В Spark таким разбиением являются партиции данных. Каждая партиция обрабатывается задачей, а задачи выполняются executor-процессами на рабочих узлах.

Ключевое отличие Spark заключается в активном использовании оперативной памяти. Если данные многократно используются в цепочке преобразований, их можно кэшировать, снижая количество обращений к диску и ускоряя последующие вычисления. Это делает Spark эффективным для интерактивной аналитики, итеративных алгоритмов и повторного использования промежуточных результатов.

Производительность Spark зависит от локальности данных. Чем ближе задача выполняется к месту хранения соответствующей партиции, тем меньше сетевые задержки. В конспекте рассматриваются уровни локальности `PROCESS_LOCAL`, `NODE_LOCAL`, `RACK_LOCAL` и `ANY`, отражающие степень близости вычислений к данным.

### **2.2. Модель RDD и механизм lineage**

RDD, или Resilient Distributed Dataset, представляет собой неизменяемый распределённый набор данных, разделённый на партиции. RDD не хранит всю информацию в драйвере; он содержит метаданные о партициях, зависимостях от родительских RDD, функции вычисления и предпочтительных местах размещения данных.

Неизменяемость RDD означает, что любая операция преобразования создаёт новый набор данных, а не изменяет существующий. Это свойство

упрощает восстановление после сбоев: Spark запоминает lineage - цепочку операций, из которой можно повторно вычислить потерянную партицию без полного копирования всех промежуточных данных.

В RDD API различают transformations и actions. Transformations, например map(), filter(), flatMap() и reduceByKey(), формируют новый RDD и выполняются лениво. Actions, например count(), collect(), mean(), sum(), top() или saveAsTextFile(), запускают фактическое выполнение графа вычислений и возвращают результат в драйвер либо записывают его во внешнее хранилище.

### **2.3. DataFrame, Spark SQL и оптимизация запросов**

DataFrame в PySpark является высокоуровневой табличной абстракцией поверх распределённых данных. В отличие от RDD, DataFrame имеет схему, которая описывает имена столбцов и типы данных. Наличие схемы позволяет Spark анализировать выражения, оптимизировать план выполнения и эффективнее использовать память.

Spark SQL и DataFrame API используют оптимизатор Catalyst. Он преобразует логический план запроса, применяет правила оптимизации и выбирает физический план выполнения. Для разработчика это означает, что многие операции - фильтрации, проекции, агрегации, joins - могут быть выполнены эффективнее, чем эквивалентные ручные преобразования на уровне RDD.

Внутренний механизм Tungsten направлен на эффективное использование CPU и памяти: Spark применяет бинарное представление данных, генерацию кода и оптимизированные структуры выполнения. Благодаря этому DataFrame API обычно предпочтителен для аналитических задач, где данные имеют понятную структуру и могут быть описаны схемой.

## **2.4. Узкие и широкие преобразования, shuffle и партиционирование**

Преобразования Spark делятся на узкие и широкие. Узкие преобразования, такие как `map()` и `filter()`, позволяют вычислить выходную партицию на основе одной или ограниченного числа входных партиций. Они хорошо масштабируются и не требуют массового обмена данными между узлами.

Широкие преобразования, например `groupByKey()`, `reduceByKey()`, `join()` и некоторые виды сортировки, требуют перераспределения данных по ключам. Этот процесс называется `shuffle` и является одной из самых дорогих операций в Spark, поскольку включает сетевую передачу, запись временных данных и синхронизацию между стадиями выполнения.

Партиционирование определяет, как данные распределены по узлам и задачам. Правильный выбор числа партиций и ключей группировки снижает риск перекоса данных и повышает эффективность параллелизма. В практической части используется паттерн `reduceByKey()` для распределённой агрегации, который предпочтительнее `driver-side` подсчётов на больших наборах данных.

## **2.5. Надёжность, кэширование и контрольные точки**

Отказоустойчивость Spark строится на `lineage` и автоматическом перезапуске задач. Если `executor` теряет часть данных, Spark может восстановить соответствующие партиции на основе исходных данных и цепочки преобразований. Такой подход снижает необходимость в полном дублировании промежуточных результатов.

Кэширование применяется, когда один и тот же `DataFrame` или `RDD` используется повторно. В этом случае Spark сохраняет результат в памяти или на диске, что уменьшает стоимость повторных `actions`. Однако кэширование

требует контроля объёма памяти: избыточное сохранение промежуточных наборов может вытеснять полезные данные и ухудшать производительность.

Checkpointing фиксирует состояние данных в надёжном хранилище и обрывает длинную цепочку lineage. Этот механизм полезен для сложных итеративных вычислений, потоковой обработки и сценариев, где восстановление по длинной цепочке преобразований становится слишком дорогим.

### 3. ИССЛЕДОВАНИЕ ИНСТРУМЕНТА PYSPARK

#### 3.1. Архитектура Spark-приложения

Spark-приложение состоит из driver-процесса, cluster manager и executor-процессов. Driver управляет жизненным циклом приложения, создаёт SparkSession, формирует DAG и распределяет задачи. Cluster manager выделяет вычислительные ресурсы. Executors выполняют задачи, хранят кэшированные данные и участвуют в shuffle.

В локальном режиме `local[*]`, который используется в учебной практике и Google Colab, Spark имитирует кластер на одной машине, задействуя доступные ядра процессора. Несмотря на локальное выполнение, логика работы остаётся близкой к кластерной: создаются задачи, формируются стадии, применяются партиции и сохраняется ленивое выполнение.

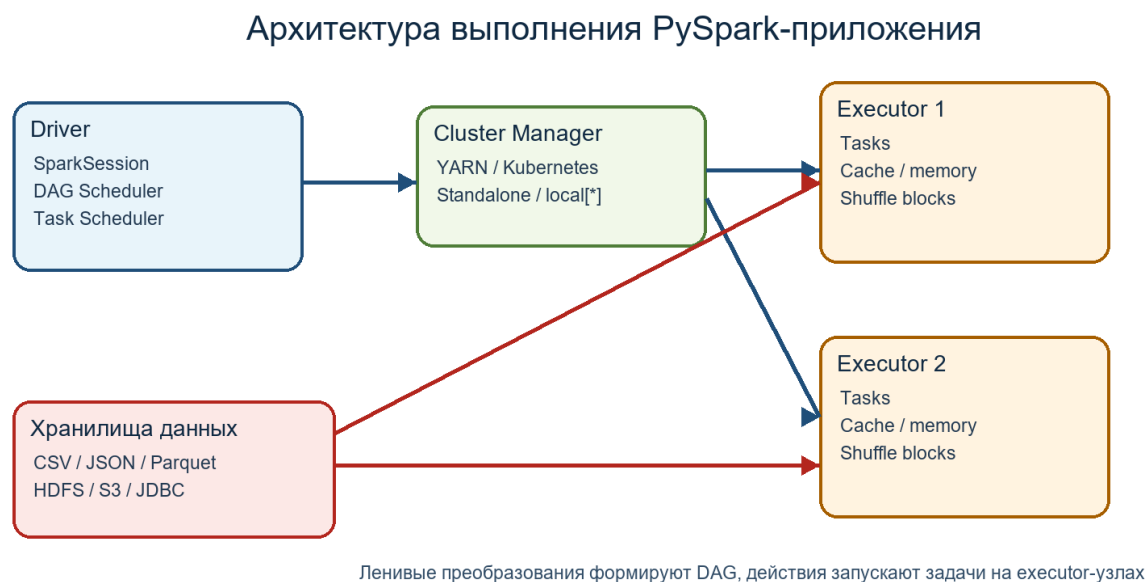


Рисунок 1 - Упрощённая архитектура выполнения PySpark-приложения

#### 3.2. Основные компоненты и абстракции

PySpark предоставляет несколько уровней работы с данными: низкоуровневый RDD API, табличный DataFrame API, SQL-интерфейс, а также

специализированные библиотеки Spark Streaming, MLlib и GraphX в экосистеме Spark. На практике выбор API зависит от структуры данных, требований к оптимизации и необходимости низкоуровневого контроля.

Таблица 1 - Ключевые абстракции PySpark

| Абстракция     | Назначение                                | Практическое применение                                                                         |
|----------------|-------------------------------------------|-------------------------------------------------------------------------------------------------|
| SparkSession   | Единая точка входа в приложение Spark     | Создание DataFrame, чтение файлов, выполнение SQL-запросов                                      |
| SparkContext   | Доступ к низкоуровневому Spark Core       | Создание RDD через <code>parallelize()</code> и <code>textFile()</code>                         |
| RDD            | Неизменяемый распределённый набор данных  | Парсинг логов, распределённые <code>map/filter/reduceByKey</code>                               |
| DataFrame      | Распределённая таблица со схемой          | Агрегации, <code>joins</code> , оконные функции, работа с вложенными типами                     |
| Transformation | Ленивое преобразование данных             | <code>map()</code> , <code>filter()</code> , <code>withColumn()</code> , <code>groupBy()</code> |
| Action         | Операция, запускающая выполнение          | <code>count()</code> , <code>show()</code> , <code>collect()</code> , <code>write()</code>      |
| Partition      | Логическая часть распределённого набора   | Единица параллельного выполнения задач                                                          |
| Shuffle        | Перераспределение данных между партициями | <code>groupBy</code> , <code>join</code> , сортировки и широкие преобразования                  |

### 3.3. Работа со схемами и сложными типами данных

Одним из преимуществ DataFrame API является явное описание структуры данных. В PySpark для этого используются типы `StructType`,

StructField, StringType, IntegerType, DoubleType, BooleanType и другие. Схема делает обработку более предсказуемой, позволяет контролировать nullable-поля и снижает риск ошибок при преобразовании типов.

В практической работе используются сложные типы ArrayType и MapType. MapType удобен для представления количества пользовательских сессий по устройствам, а ArrayType - для списков посещённых страниц или участников организаций. Функции flatten(), array\_distinct(), collect\_set(), split() и explode\_outer() позволяют преобразовывать вложенные структуры в аналитически пригодный вид.

Оконные функции Window применяются для задач, где требуется сравнение строки с соседними значениями или ранжирование внутри упорядоченной выборки. В анализе финансовых данных АвтоВАЗа через lag() рассчитывается прибыль предыдущего года, а через rank() определяется ранг периода по величине изменения прибыли.

### **3.4. Форматы данных и интеграции**

Spark поддерживает чтение и запись множества форматов данных, включая CSV, JSON, Parquet, ORC и Avro. Для аналитических задач особенно важен Parquet, поскольку он хранит данные в колоночном формате, поддерживает схему и хорошо подходит для выборочного чтения столбцов.

PySpark также интегрируется с JDBC-источниками, распределёнными файловыми системами и облачными хранилищами. Это позволяет использовать его как универсальный слой обработки между источниками данных, хранилищами, витринами и инструментами аналитики.



## 4. ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ ОБРАБОТКИ ДАННЫХ

Практическая часть работы основана на материалах каталога PySparkPractice. Проект содержит два основных направления: RDD/PySpark\_rdd.ipynb для отработки низкоуровневых распределённых преобразований и DataFrame/PySparkDataFrame.ipynb для решения аналитических задач средствами DataFrame API. Дополнительно используется конспект spark.docx, в котором систематизированы теоретические сведения об архитектуре Spark, RDD, DataFrame, shuffle, партиционировании, join-стратегиях, кэшировании и оптимизаторе Catalyst.

Среда выполнения практики ориентирована на Jupyter/Google Colab. Для запуска устанавливаются зависимости pyspark и py4j, создаётся SparkSession, а данные загружаются из файлов или формируются в памяти. Такой формат удобен для исследования, поскольку позволяет последовательно выполнять ячейки, смотреть схемы, промежуточные таблицы и результаты агрегирования.

Таблица 2 - Структура практической реализации

| Направление     | Датасеты и задачи                                                             | Использованные возможности PySpark                                                            |
|-----------------|-------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| RDD API         | Логи сенсоров, пароли, русский словарь, веб-логи, посты соцсети, список стран | parallelize(), textFile(), map(), filter(), reduceByKey(), distinct(), count(), mean(), top() |
| DataFrame API   | Онлайн-активность пользователей, финансы АвтоВАЗа, данные Star Wars           | SparkSession, схемы, MapType, ArrayType, withColumn(), groupBy(), Window, joins, Parquet      |
| Оптимизация     | Снижение нагрузки на драйвер и контроль shuffle                               | reduceByKey(), ограничение collect(), кэширование, explain(), выбор join-стратегий            |
| Качество данных | NULL-значения, дубликаты, неоднородные поля и нормализация схемы              | coalesce(), dropDuplicates(), приведение типов, стандартизация имён столбцов                  |

#### 4.1. Реализация аналитики на RDD

RDD-направление демонстрирует работу с неструктурированными и полуструктурированными данными. В блоке с логами промышленных сенсоров строки разделяются по символу '|', затем значения температуры и влажности приводятся к числовым типам с явной обработкой NULL. На этой основе рассчитываются распределение статусов, частота ошибок по сенсорам, средняя температура и признаки потенциальных аномалий.

При анализе распространённых паролей используется внешний текстовый файл объёмом более 681 тыс. записей. Рассчитываются длины паролей, символьный состав, частые префиксы и суффиксы. Важным методическим моментом является использование `reduceByKey()` для распределённого подсчёта суффиксов вместо переноса больших частотных словарей на драйвер.

Лингвистический анализ русского словаря показывает, как RDD API применяется к текстовым данным. Рассчитывается распределение слов по длинам, определяется наиболее частая длина слова, подсчитываются слова с буквой 'ё' и фильтруются палиндромы. Этот сценарий хорошо демонстрирует `map/filter/reduce`-подход для обработки строк.

Веб-логи в формате Apache требуют парсинга регулярным выражением и выделения IP-адреса, HTTP-метода, эндпоинта, статус-кода и размера ответа. Далее рассчитываются общее число запросов, уникальные IP, средний размер ответа, распределение кодов статуса, доля успешных запросов и наиболее популярные эндпоинты.

Для постов социальной сети строятся метрики вовлечённости: суммарные лайки и комментарии, средние значения на пост, распределение по типам контента, топ постов по лайкам и комментариям. Датасет стран используется для простого текстового анализа: первая буква, длина названия, многословные названия и средняя длина.

Таблица 3 - Основные результаты RDD-практики

| Сценарий        | Ключевые результаты                                                              | Методический смысл                                        |
|-----------------|----------------------------------------------------------------------------------|-----------------------------------------------------------|
| Логи сенсоров   | 28 OK, 6 WARNING и 6 ERROR записей из 40 строк;<br>обнаружение ошибок и аномалий | Парсинг строк, обработка NULL, reduceByKey()              |
| Пароли          | Средняя длина 9 символов; 441 347 числовых и 115 088 буквенных паролей           | Распределённый частотный анализ больших текстовых наборов |
| Русский словарь | 51 301 слово; пик длины на 8 символах; 36 палиндромов                            | Лингвистические фильтры и агрегирование по ключам         |
| Веб-логи        | 10 000 запросов; HTTP 200 - 38,5%; топ эндпоинт /search?q=spark                  | Regex-парсинг логов и HTTP-аналитика                      |
| Посты соцсети   | 707 567 лайков и 8 387 комментариев; лучшие показатели у Carousel/Reel           | Агрегации и ранжирование пользовательского контента       |
| Страны          | Средняя длина названия 8,5 символа; 41 многословная запись                       | Текстовый анализ справочного набора                       |

## 4.2. Реализация аналитики на DataFrame

DataFrame-направление показывает преимущества структурированного API. В первом блоке создаётся DataFrame пользовательской активности со сложными типами: карта device\_type хранит количество сессий по устройствам, а массив page\_views хранит посещённые страницы. Далее через withColumn()

рассчитываются `mobile_sessions` и `total_sessions_count`, а через `groupBy()` строится агрегированная витрина по пользователям.

В анализе финансовых данных АвтоВАЗа демонстрируется предварительная подготовка CSV-файла: нормализация названий столбцов, использование числовых типов и построение аналитических колонок. Оконная функция `lag()` позволяет сравнить чистую прибыль с предыдущим годом, а `rank()` используется для ранжирования периодов по абсолютному изменению прибыли. Дополнительно через `when()` формируются категории прибыльности.

Блок Star Wars использует три Parquet-датасета: `species`, `organizations` и `characters`. После удаления дубликатов выполняются `joins`, подсчёт персонажей по видам и классификациям, расчёт среднего роста, преобразование строковых списков лидеров и участников в массивы, разворачивание массивов через `explode_outer()`, а также расчёт индекса массы тела персонажей.

Практика DataFrame подтверждает, что высокоуровневый API удобен для аналитических задач со схемой: он сокращает объём ручного парсинга, упрощает работу с вложенными типами и позволяет Spark применять оптимизации физического плана.

### **4.3. Паттерны производительности и качества**

Ключевой паттерн производительности в RDD API - избегать сбора больших наборов данных на драйвер. Операции `collect()` и `countByValue()` удобны для небольших данных, но при росте объёма могут привести к нехватке памяти. Поэтому для больших частотных подсчётов предпочтительны `reduceByKey()`, `aggregateByKey()` и другие распределённые агрегаторы.

В DataFrame API важны корректные схемы и минимизация лишних преобразований. Функция `coalesce()` применяется для безопасной обработки отсутствующих ключей в `map`-полях, `dropDuplicates()` - для очистки

справочников, а явное приведение типов снижает риск некорректных вычислений. Для анализа производительности используется `df.explain()`, позволяющий увидеть физический план и операции `shuffle`.

На уровне архитектуры следует различать сценарии, где лучше использовать RDD, и сценарии, где предпочтителен DataFrame. RDD полезен при низкоуровневом контроле, нестандартном парсинге и работе с произвольными объектами. DataFrame эффективнее для табличной аналитики, агрегаций, оконных функций, `joins` и форматов со схемой.

## 5. ПРЕИМУЩЕСТВА И ОГРАНИЧЕНИЯ PYSPARK

Главным преимуществом PySpark является масштабируемость. Один и тот же код может выполняться в локальном режиме для обучения и в кластерном режиме для обработки больших наборов данных. Распределение данных по партициям и выполнение задач на executor-узлах позволяют обрабатывать объёмы, которые не помещаются в память одного процесса.

Второе важное преимущество - единая платформа. Spark объединяет пакетную обработку, SQL-аналитику, потоковую обработку, машинное обучение и работу с разными форматами данных. Это снижает количество разрозненных инструментов в проекте и упрощает перенос логики между исследовательской и промышленной средой.

Третье преимущество связано с выразительностью API. Разработчик может использовать RDD для низкоуровневого контроля, DataFrame для оптимизируемой табличной аналитики и SQL для декларативных запросов. Такая гибкость делает PySpark удобным как для дата-инженерии, так и для аналитических исследований.

К ограничениям PySpark относится накладная стоимость взаимодействия между Python и JVM. Особенно заметна она при использовании пользовательских Python UDF, когда данные необходимо сериализовать и передавать между средами выполнения. Поэтому в производительных конвейерах предпочтительно использовать встроенные функции `r pyspark.sql.functions`.

Ещё одно ограничение - сложность диагностики распределённых ошибок. Проблемы с памятью, перекосом ключей, неудачным shuffle или неэффективным join часто проявляются только на больших объёмах данных. Для их выявления требуется анализ Spark UI, физического плана, логов executor-узлов и структуры партиций.

PySpark также не всегда оправдан для малых данных. Если набор помещается в память одного процесса и не требует распределённого выполнения, pandas может оказаться проще и быстрее из-за меньших накладных расходов. Поэтому выбор PySpark должен быть связан с реальными требованиями к масштабу, отказоустойчивости и интеграции с инфраструктурой больших данных.

## 6. СРАВНЕНИЕ С ДРУГИМИ ИНСТРУМЕНТАМИ

PySpark следует сравнивать не только с библиотеками Python, но и с системами распределённой обработки. Для локального анализа часто используется pandas, для масштабирования Python-вычислений - Dask, для потоковой обработки - Apache Flink, а для классической пакетной обработки - Hadoop MapReduce. Каждый инструмент имеет собственную область применимости.

Таблица 4 - Сравнение PySpark с альтернативными инструментами

| Параметр             | PySpark                           | pandas              | Dask                           | Apache Flink          | Hadoop MapReduce        |
|----------------------|-----------------------------------|---------------------|--------------------------------|-----------------------|-------------------------|
| Тип обработки        | Batch, SQL, streaming, ML         | Локальная аналитика | Параллельные Python-вычисления | Потоковая и batch     | Batch                   |
| Масштабирование      | Кластерное                        | Один процесс        | Локальное и кластерное         | Кластерное            | Кластерное              |
| Основная абстракция  | RDD, DataFrame, SQL               | DataFrame           | Dask DataFrame/Array           | DataStream/Table      | Map и Reduce            |
| Оптимизация запросов | Catalyst, Tungsten                | Ограниченная        | Зависит от графа Dask          | Оптимизатор Table API | Минимальная             |
| Работа с памятью     | In-memory + spill                 | Память процесса     | Out-of-core                    | State management      | Диск между стадиями     |
| Порог входа          | Средний                           | Низкий              | Средний                        | Высокий               | Средний                 |
| Лучшие сценарии      | ETL, большие таблицы, joins, логи | EDA малых данных    | Python-задачи с параллелизмом  | Real-time события     | Простая batch-обработка |



|             |                                       |                                 |                                |                        |                         |
|-------------|---------------------------------------|---------------------------------|--------------------------------|------------------------|-------------------------|
| Ограничения | Накладные расходы, сложность кластера | Не масштабируется горизонтально | Меньше промышленных интеграций | Сложность эксплуатации | Менее выразительный API |
|-------------|---------------------------------------|---------------------------------|--------------------------------|------------------------|-------------------------|

По сравнению с pandas, PySpark выигрывает при больших объёмах данных, необходимости чтения распределённых файловых форматов и выполнении тяжёлых агрегаций. Однако pandas остаётся удобнее для малых наборов, быстрых локальных экспериментов и визуального анализа.

По сравнению с Dask, PySpark имеет более зрелую экосистему для ETL, SQL и интеграции с хранилищами больших данных. Dask удобен, когда нужно масштабировать существующий Python-код и сохранить близость к NumPy/pandas, но Spark чаще выбирается для промышленных data pipeline.

Apache Flink сильнее ориентирован на потоковую обработку с низкими задержками и управлением состоянием. Spark Structured Streaming также поддерживает streaming-сценарии, но исторически Spark особенно силён в batch-обработке, интерактивной аналитике и SQL-ориентированных задачах.

## 7. СОВМЕСТИМОСТЬ PYSPARK С ЭКОСИСТЕМОЙ ДАННЫХ

PySpark редко используется изолированно. В реальных проектах он становится частью более широкой платформы, где данные загружаются из разных источников, обрабатываются в распределённой среде, сохраняются в оптимизированных форматах и затем используются аналитиками, ML-моделями или BI-системами.

Таблица 5 - Совместимость PySpark с инструментами и форматами

| Категория                   | Примеры                                     | Роль в PySpark-проектах                                |
|-----------------------------|---------------------------------------------|--------------------------------------------------------|
| Хранилища                   | HDFS, S3, ADLS, локальная файловая система  | Чтение и запись больших наборов данных                 |
| Форматы                     | CSV, JSON, Parquet, ORC, Avro               | Обмен данными и оптимизация хранения                   |
| Табличные lakehouse-форматы | Delta Lake, Apache Iceberg, Apache Hudi     | ACID-таблицы, инкрементальные обновления, time travel  |
| Оркестрация                 | Apache Airflow, Prefect                     | Запуск Spark-задач по расписанию и в составе DAG       |
| Инфраструктура              | YARN, Kubernetes, Standalone                | Выделение ресурсов и управление кластером              |
| Потоки событий              | Kafka, файловые потоки                      | Structured Streaming и near-real-time обработка        |
| ML и эксперименты           | MLlib, MLflow, scikit-learn                 | Подготовка признаков, обучение и сопровождение моделей |
| Среды разработки            | Jupyter, Google Colab, Databricks notebooks | Интерактивная разработка и исследование данных         |

Практический проект PySparkPractice демонстрирует учебный вариант такой совместимости: код выполняется в ноутбуках, данные читаются из текстовых, CSV и Parquet-файлов, а обработка строится вокруг SparkSession и SparkContext. При переносе в промышленную среду эти же принципы могут быть расширены за счёт оркестрации, кластерного менеджера, lakehouse-форматов и систем мониторинга.

Особенно перспективным направлением является сочетание PySpark с Delta Lake, Iceberg или Hudi. Такие форматы позволяют хранить большие аналитические таблицы с поддержкой транзакций, схем, инкрементальных изменений и версионирования. Для исследовательского проекта это может стать следующим шагом после базовой практики RDD и DataFrame.

## ЗАКЛЮЧЕНИЕ

В ходе работы был проведён анализ PySpark как инструмента распределённой обработки больших данных. Рассмотрены архитектура Apache Spark, модель выполнения driver/executor, роль cluster manager, механизм построения DAG, ленивые вычисления, партиционирование, shuffle, кэширование и контрольные точки.

Исследование показало, что PySpark предоставляет два взаимодополняющих уровня работы с данными. RDD API позволяет понять фундаментальные принципы распределённой обработки, управлять преобразованиями на уровне элементов и реализовывать нестандартный парсинг. DataFrame API предоставляет более высокий уровень абстракции, использует схемы, встроенные функции, Catalyst Optimizer и лучше подходит для структурированной аналитики.

Практическая часть подтвердила применимость PySpark к различным доменам данных: промышленным логам, паролям, словарям, веб-журналам, социальным метрикам, справочникам стран, пользовательской активности, финансовым показателям и Parquet-датасетам. Были использованы map(), filter(), reduceByKey(), groupBy(), withColumn(), Window, joins, dropDuplicates(), explode\_outer() и другие операции.

Ключевой вывод состоит в том, что PySpark эффективен не только как средство запуска вычислений на больших объёмах данных, но и как методологическая основа для проектирования воспроизводимых аналитических конвейеров. При этом высокая производительность достигается только при понимании физической модели выполнения: необходимо контролировать операции на драйвере, избегать лишнего shuffle, использовать схемы, анализировать планы выполнения и выбирать подходящий уровень API.

Ограничения PySpark связаны с накладными расходами распределённой среды, сложностью настройки кластера, возможными проблемами Python UDF и необходимостью диагностики производительности. Поэтому инструмент наиболее оправдан там, где данные действительно требуют горизонтального масштабирования, а не помещаются в комфортные рамки локального анализа.

В дальнейшем проект может быть развит за счёт добавления Spark Structured Streaming, Delta Lake или Iceberg, автоматизированных тестов качества данных, визуализации результатов, MLlib-конвейеров и запуска практических сценариев в полноценной кластерной среде.

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Apache Spark Documentation. Spark Overview. URL: <https://spark.apache.org/docs/latest/>
2. PySpark API Reference. URL: <https://spark.apache.org/docs/latest/api/python/>
3. Zaharia M., Chowdhury M., Franklin M. J., Shenker S., Stoica I. Spark: Cluster Computing with Working Sets. HotCloud, 2010.
4. Zaharia M., Chowdhury M., Das T. et al. Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing. NSDI, 2012.
5. Armbrust M., Xin R. S., Lian C. et al. Spark SQL: Relational Data Processing in Spark. SIGMOD, 2015.
6. Karau H., Warren R. High Performance Spark. O'Reilly Media, 2017.
7. Damji J., Wenig B., Das T., Lee D. Learning Spark: Lightning-Fast Data Analytics. 2nd ed. O'Reilly Media, 2020.
8. Материалы практической работы PySparkPractice: ноутбуки RDD/PySpark\_rdd.ipynb и DataFrame/PySparkDataFrame.ipynb.
9. Конспект spark.docx по архитектуре Apache Spark, RDD, DataFrame, shuffle, partitioning, Catalyst и Tungsten.